

MARMARA UNIVERSITY  
Faculty of Engineering  
Department of Electrical and Electronics Engineering



MARMARA  
UNIVERSITY

EE1004 – Object-Oriented Programming  
Group Project (Project 14)

---

# Cargo Shipment Tracker

An Object-Oriented Console Application in Java

---

## Group 14 – Members

|                     |           |
|---------------------|-----------|
| Atabey Aydın        | 150718503 |
| Mehmet Açar         | 150719020 |
| İsmail Hanifi Nal   | 150719025 |
| Abdulkadir Köroğlu  | 150719695 |
| Burak Gökmen        | 150720010 |
| Alperen Tufan Pelit | 150720012 |

## Supervision

Course Lecturer: Assoc. Prof. Dr. Salih Bayar  
Laboratory Assistant: Res. Asst. Salih Çolakoğlu

Spring 2025–2026  
June 2026

# Cargo Shipment Tracker: An Object-Oriented Console Application in Java

EE1004 Object-Oriented Programming – Group Project 14

|                                                                 |                                                               |                                                                      |
|-----------------------------------------------------------------|---------------------------------------------------------------|----------------------------------------------------------------------|
| <b>Atabey Aydın</b><br>150718503<br>atabeyaydi@marun.edu.tr     | <b>Mehmet Açar</b><br>150719020<br>mehmetacar19@marun.edu.tr  | <b>İsmail Hanifi Nal</b><br>150719025<br>ismailnal@marun.edu.tr      |
| <b>Abdulkadir Köroğlu</b><br>150719695<br>akoroglu@marun.edu.tr | <b>Burak Gökmen</b><br>150720010<br>burak.gokmen@marun.edu.tr | <b>Alperen Tufan Pelit</b><br>150720012<br>alperenpelit@marun.edu.tr |

**Abstract**—This report presents the design and implementation of a *Cargo Shipment Tracker*, a console-based Java application developed for the EE1004 group project. The system models three shipment classes—standard, express, and same-day—using an abstract base class, three concrete subclasses, and a separate **Insurable** interface, exercising the four pillars of object-oriented programming together with interface-based capability decoupling. Shipping cost is computed polymorphically as the product of distance (km) and a per-class per-kilometre rate; insurance liability is reported as a class-specific percentage of cost. The shipment life cycle (PENDING → IN\_TRANSIT → DELIVERED or RETURNED; PENDING may also short-cut to RETURNED) is enforced by an **enum**-based state machine that raises a custom unchecked exception (**InvalidStatusTransitionException**) on every illegal transition. A **CargoCompany** aggregator stores shipments in both an **ArrayList** (insertion order) and a **HashMap** (O(1) lookup by tracking ID, as mandated by the specification), and exposes three interchangeable **Comparator** sorting strategies (by cost, by distance, by status). The application is fully menu-driven, validates all user input, and produces locale-correct numeric output suitable for automated grading. Testing against the specification’s sample I/O confirms an exact match.

**Index Terms**—object-oriented programming, abstract class, interface, inheritance, polymorphism, encapsulation, enum state machine, custom exception, Java collections, **HashMap**, **Comparator**, **Strategy** pattern.

## Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                              | <b>3</b>  |
| 1.1      | Project Overview . . . . .                                       | 3         |
| 1.2      | Scenario . . . . .                                               | 3         |
| 1.3      | Objectives . . . . .                                             | 3         |
| 1.4      | Development Environment . . . . .                                | 3         |
| <b>2</b> | <b>Requirements Analysis</b>                                     | <b>3</b>  |
| 2.1      | Functional Requirements . . . . .                                | 3         |
| 2.2      | Non-Functional Requirements . . . . .                            | 4         |
| <b>3</b> | <b>System Design</b>                                             | <b>5</b>  |
| 3.1      | Class Architecture . . . . .                                     | 5         |
| 3.2      | Class Responsibilities and Pricing . . . . .                     | 5         |
| 3.3      | OOP Pillars and Interface Decoupling . . . . .                   | 7         |
| 3.4      | Design Rationale . . . . .                                       | 7         |
| 3.5      | Shipment Life-Cycle State Machine . . . . .                      | 9         |
| <b>4</b> | <b>Implementation</b>                                            | <b>9</b>  |
| 4.1      | Interface and Abstract Class . . . . .                           | 9         |
| 4.2      | Enum State Machine . . . . .                                     | 10        |
| 4.3      | Custom Exception . . . . .                                       | 11        |
| 4.4      | Aggregator with Dual Storage and Comparator Strategies . . . . . | 11        |
| 4.5      | Menu-Driven Main Loop . . . . .                                  | 12        |
| <b>5</b> | <b>Testing and Results</b>                                       | <b>13</b> |
| 5.1      | Specification Walkthrough . . . . .                              | 13        |
| 5.2      | Numeric Verification . . . . .                                   | 13        |
| 5.3      | Edge-Case Test Results . . . . .                                 | 13        |
| <b>6</b> | <b>Limitations and Future Work</b>                               | <b>13</b> |
| 6.1      | Current Limitations . . . . .                                    | 13        |
| 6.2      | Potential Extensions . . . . .                                   | 14        |
| <b>7</b> | <b>Conclusion</b>                                                | <b>14</b> |
|          | <b>References</b>                                                | <b>15</b> |
| <b>A</b> | <b>Console Transcript (Full Run)</b>                             | <b>15</b> |
| <b>B</b> | <b>Member Contributions</b>                                      | <b>18</b> |
| <b>C</b> | <b>OOP Concept Self-Assessment</b>                               | <b>19</b> |
| <b>D</b> | <b>Pre-Submission Checklist</b>                                  | <b>20</b> |
| <b>E</b> | <b>GitHub Repository Information</b>                             | <b>20</b> |
| <b>F</b> | <b>AI-Use Disclosure</b>                                         | <b>20</b> |

# 1 Introduction

## 1.1 Project Overview

This project implements a *Cargo Shipment Tracker*: a command-line Java program that registers cargo shipments at an Istanbul depot, tracks each shipment through a strict delivery life cycle, computes shipping cost from a distance-based tariff, and lets the operator search, list, sort, and summarise the daily roster. The application is designed as a teaching vehicle for object-oriented design and therefore exercises every core OOP construct required by the EE1004 project specification [1]: an abstract base class, three concrete subclasses, an interface, a Java `enum`, a custom exception, multiple `Comparator` strategies, and `HashMap`-based fast lookup.

## 1.2 Scenario

The customer for the system is Mehmet, an operations coordinator at a small but growing cargo company in Istanbul. Each morning shipments arrive at the depot with different priority classes—standard, express, and same-day—each with its own per-kilometre rate, its own weight cap, and its own insurance percentage. Once a shipment is registered it follows a strict lifecycle: it starts as `PENDING`, moves to `IN_TRANSIT` when the driver picks it up, and ends as either `DELIVERED` or `RETURNED`. Mehmet wants a Java application that captures all of this cleanly so that he can stop relying on hand-written log sheets.

## 1.3 Objectives

The concrete objectives of the project are:

- to model a realistic logistics domain with a clean class hierarchy and an interface for the cross-cutting insurance concern;
- to demonstrate abstraction, inheritance, polymorphism, encapsulation, and interface-based capability decoupling in a single coherent codebase;
- to enforce business rules (status transitions, weight caps) with type-safe constructs (an `enum` state machine and a custom unchecked exception);
- to provide efficient data access using the Java collections framework (mandatory `HashMap` for  $O(1)$  lookup by tracking ID); and
- to deliver a robust, menu-driven interface that never crashes on bad input and emits locale-correct output for automated grading.

## 1.4 Development Environment

Table 1 summarises the environment used to build and run the application.

# 2 Requirements Analysis

## 2.1 Functional Requirements

Table 2 lists the functional requirements (FR) the application fulfils. These correspond directly to the nine main-menu options demanded by the specification.

**Table 1:** Development environment

| Item               | Value                                       |
|--------------------|---------------------------------------------|
| Language           | Java (OpenJDK 11 compatible)                |
| Source files       | 9 <code>.java</code> files (single package) |
| Build command      | <code>javac *.java</code>                   |
| Run command        | <code>java Main</code>                      |
| External libraries | None (Java Standard Edition only)           |
| Character encoding | UTF-8                                       |
| Numeric locale     | <code>Locale.US</code> (decimal point)      |

**Table 2:** Functional requirements

| ID    | Requirement                                                                                                                                                                                                           |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-01 | Register a new shipment of type standard, express, or same-day, capturing sender, recipient, distance in km, and weight in kg.                                                                                        |
| FR-02 | Assign every shipment a unique, sequential integer tracking ID automatically (static counter shared by all subclasses).                                                                                               |
| FR-03 | Refuse registration when <code>weightKg</code> exceeds the per-class cap; print the exact message <code>Weight exceeds capacity for this shipment class. Shipment not registered.</code> and do not store the object. |
| FR-04 | Advance a shipment's status, permitting only the legal life-cycle transitions; raise <code>InvalidStatusTransitionException</code> on any illegal request.                                                            |
| FR-05 | List all registered shipments in insertion order (polymorphic <code>toString()</code> ).                                                                                                                              |
| FR-06 | Find a single shipment by its tracking ID using <code>HashMap.get()</code> (no linear search). When found, also display its insurance line.                                                                           |
| FR-07 | Sort and display shipments by cost (descending), distance (descending), or status (lifecycle order: <code>PENDING</code> first, terminal states last).                                                                |
| FR-08 | Display a revenue & insurance summary giving the total number of shipments, total revenue, total insurance, and a per-class insurance breakdown.                                                                      |
| FR-09 | Exit the application cleanly on operator request, printing <code>Exiting. Goodbye!</code> .                                                                                                                           |

## 2.2 Non-Functional Requirements

Table 3 lists the non-functional requirements (NFR), most of which are imposed verbatim by the project specification and are checked by the auto-grader.

**Table 3:** Non-functional requirements

| ID     | Requirement                                                                                                                                                                                                               |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-01 | The program must never terminate abnormally on invalid input.                                                                                                                                                             |
| NFR-02 | All risky parsing operations ( <code>Double.parseDouble</code> , <code>Integer.parseInt</code> , <code>Enum.valueOf</code> ) are wrapped in <code>try/catch</code> blocks; a friendly skip message is printed on failure. |
| NFR-03 | Unknown menu options print <code>Unknown option. Skipping...</code> and the menu loops.                                                                                                                                   |
| NFR-04 | The custom exception message format is fixed:<br><code>Invalid status transition: &lt;FROM&gt; -&gt; &lt;TO&gt;</code> .                                                                                                  |
| NFR-05 | Numeric output uses <code>Locale.US</code> so the decimal separator is always a point, regardless of the host locale.                                                                                                     |
| NFR-06 | No magic numbers: per-km rates, weight caps, and insurance percentages are <code>private static final</code> constants in their owning subclass.                                                                          |
| NFR-07 | <code>findById(int)</code> MUST use <code>HashMap.get</code> ; a linear scan over the list is forbidden.                                                                                                                  |
| NFR-08 | The <code>Insurable</code> interface, the <code>ShipmentStatus</code> enum, and the <code>InvalidStatusTransitionException</code> class each live in their own <code>.java</code> file.                                   |

## 3 System Design

### 3.1 Class Architecture

The system is organised around an abstract `Shipment` base class that implements an `Insurable` interface, three concrete subclasses (standard, express, same-day) with class-specific constants, a status `enum` carrying the state-machine check, a custom unchecked exception, a `CargoCompany` aggregator/manager, and the `Main` entry-point class. Figure 1 (next page) shows the full UML class diagram.

### 3.2 Class Responsibilities and Pricing

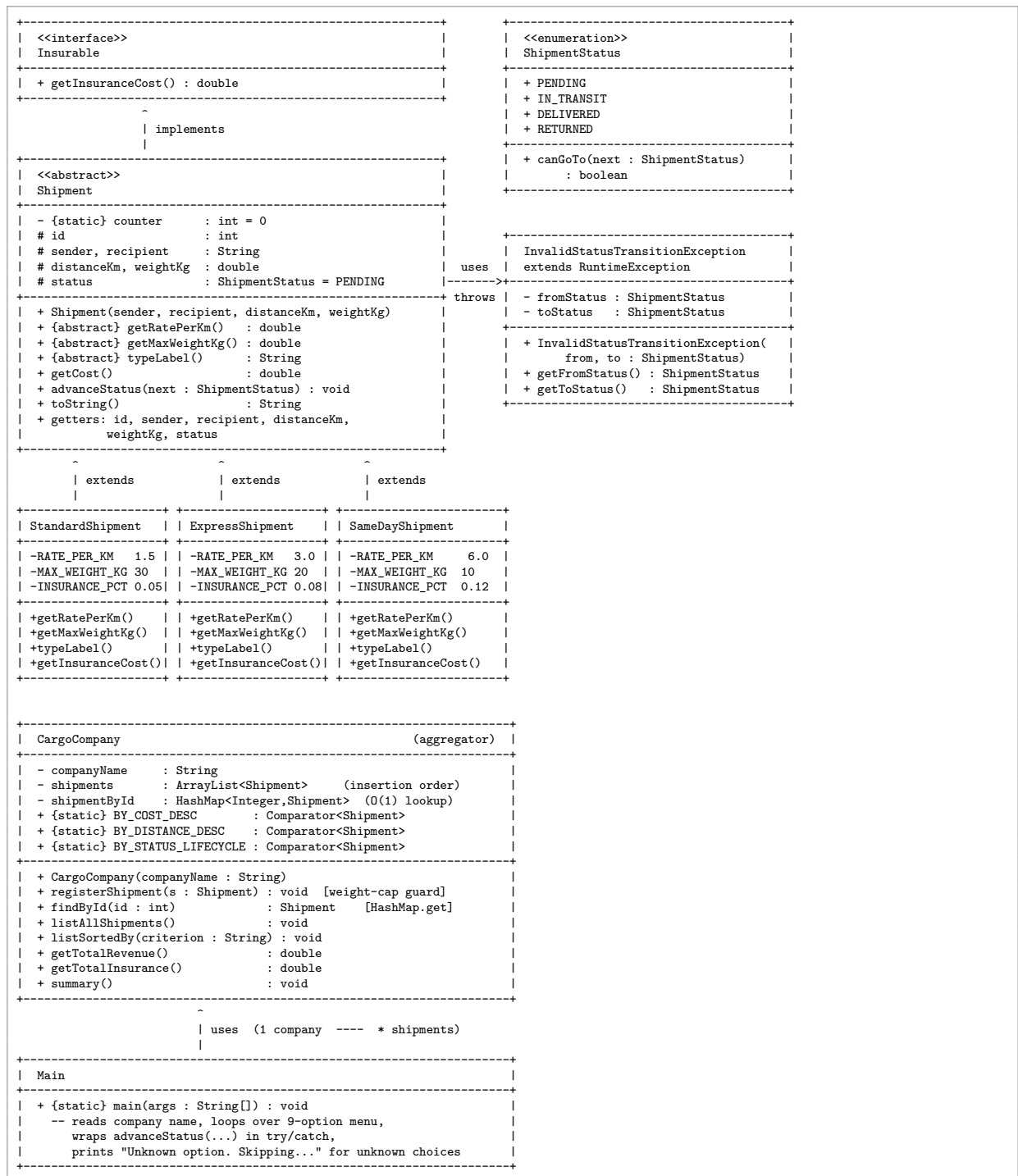
**Shipment (abstract)** holds the shared state (tracking ID, sender, recipient, distance, weight, status) and the shared concrete behaviour (cost formula, status-validated update, string rendering). It declares the abstract methods `getRatePerKm()`, `getMaxWeightKg()`, and `typeLabel()` that each subclass must implement; it implements `Insurable` but leaves `getInsuranceCost()` abstract so that each subclass applies its own percentage.

**StandardShipment**, **ExpressShipment**, **SameDayShipment** each supply their own per-kilometre rate, weight cap, and insurance percentage as `private static final` constants, and override `getInsuranceCost()` accordingly. Table 4 summarises the full pricing and insurance model.

**Table 4:** Per-class pricing, capacity, and insurance

| Type     | Class                         | Rate (TL/km) | Max weight | Insurance    |
|----------|-------------------------------|--------------|------------|--------------|
| Standard | <code>StandardShipment</code> | 1.5          | 30 kg      | 5 % of cost  |
| Express  | <code>ExpressShipment</code>  | 3.0          | 20 kg      | 8 % of cost  |
| Same-Day | <code>SameDayShipment</code>  | 6.0          | 10 kg      | 12 % of cost |

**CargoCompany** is the aggregator/manager: it holds the shipment collections (an `ArrayList` for insertion order and a `HashMap` keyed by tracking ID for  $O(1)$  lookup) and exposes the full create–read–update surface along with three named static `Comparator` constants. **Main** is the entry



point and user-interface class; it owns the menu loop and the input-validation `try/catch` blocks, and wraps every call to `advanceStatus(...)` in a `try/catch` so that the typed exception's message is printed verbatim without terminating the program.

### 3.3 OOP Pillars and Interface Decoupling

Table 5 maps each OOP principle to its concrete realisation in the code.

**Table 5:** OOP principle mapping

| Principle                 | Class / method                                                                                                                                                                 | Explanation                                                                                                                                                                                                                     |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Abstraction               | <code>Shipment</code> (abstract);<br><code>getRatePerKm()</code> ,<br><code>getMaxWeightKg()</code> , <code>typeLabel()</code> ,<br><code>getInsuranceCost()</code> (abstract) | <code>Shipment</code> cannot be instantiated; it defines the interface each subclass must fulfil without specifying how.                                                                                                        |
| Inheritance               | <code>StandardShipment</code> ,<br><code>ExpressShipment</code> ,<br><code>SameDayShipment</code> each<br>extends <code>Shipment</code>                                        | All three subclasses reuse the constructor, the static ID counter, <code>advanceStatus()</code> , <code>toString()</code> , and <code>getCost()</code> , adding only their specific rate, weight cap, and insurance percentage. |
| Polymorphism              | <code>ArrayList&lt;Shipment&gt;</code> ; over-<br>ridden <code>getRatePerKm()</code> ,<br><code>getInsuranceCost()</code> , <code>typeLabel()</code>                           | A single loop over <code>ArrayList&lt;Shipment&gt;</code> automatically invokes the correct override for each concrete type.                                                                                                    |
| Encapsulation             | <code>private/protected</code> fields;<br><code>private static final</code> constants;<br><code>advanceStatus()</code>                                                         | No external code can set an invalid weight, bypass the state machine, or mutate the tracking ID after construction.                                                                                                             |
| Interface de-<br>coupling | <code>Insurable</code> interface implemented by<br><code>Shipment</code> and overridden by every<br>subclass                                                                   | Insurance is modelled as a horizontal capability layered on top of the inheritance chain; future shipment types can opt in (or out) without restructuring the hierarchy.                                                        |

### 3.4 Design Rationale

Table 6 records the key design decisions and the reasoning behind them.

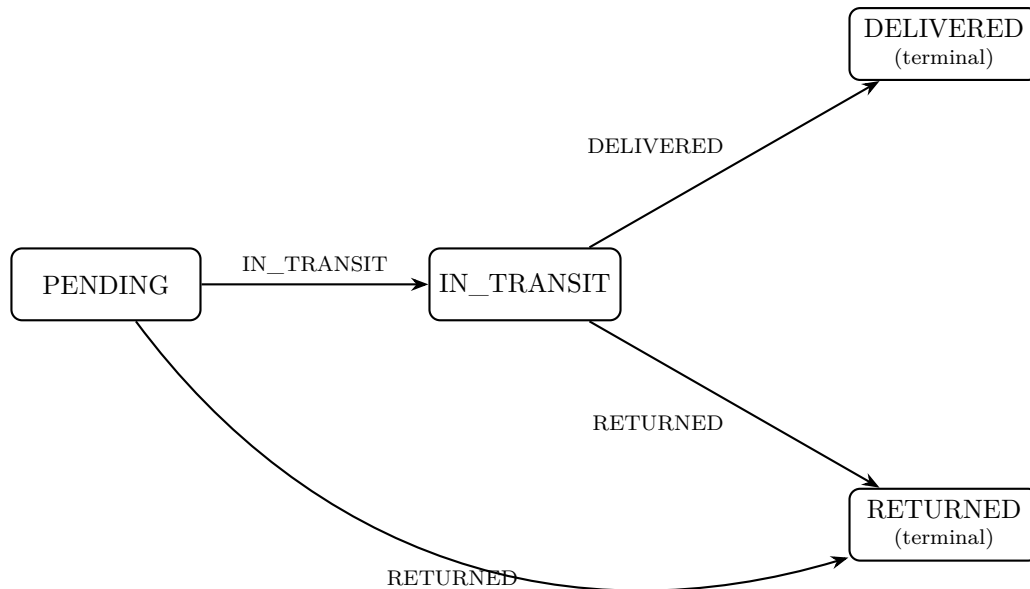


**Table 6:** Design rationale for key decisions

| Decision                                                         | Alternative                                                  | Rationale                                                                                                                                                                                                                                 |
|------------------------------------------------------------------|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use an <code>enum</code> for shipment status                     | Plain <code>String</code> constants                          | An enum gives compile-time type safety; <code>canGoTo(...)</code> switches exhaustively over the constants; a string would let any typo slip through at runtime.                                                                          |
| Encode transitions inside the enum ( <code>canGoTo</code> )      | A separate validator class                                   | Co-locating rules with states keeps them consistent; adding a state later requires editing only one file.                                                                                                                                 |
| Custom <code>InvalidStatusTransitionException</code> (unchecked) | Return a boolean; or a checked exception                     | An unchecked exception lets the violation propagate to the menu loop that knows the user context, without forcing every method on the call stack to declare <code>throws</code> ; its fields carry the exact <code>from/to</code> states. |
| <code>HashMap</code> alongside <code>ArrayList</code>            | <code>ArrayList</code> alone with linear search              | The list preserves insertion order for listing and sorting; the map gives $O(1)$ lookup for find-by-ID and status-advance, which the specification mandates explicitly.                                                                   |
| Three named <code>Comparator</code> lambdas                      | Implement <code>Comparable</code>                            | <code>Comparable</code> supports only one ordering; three interchangeable strategies (cost $\downarrow$ , distance $\downarrow$ , status lifecycle) are cleanly expressed as separate static constants—the Strategy pattern.              |
| Separate <code>Insurable</code> interface                        | A concrete <code>getInsuranceCost()</code> in the base class | The interface makes insurance an opt-in capability; a future <code>UninsuredBulkShipment</code> could extend <code>Shipment</code> without inheriting the insurance contract.                                                             |

### 3.5 Shipment Life-Cycle State Machine

Figure 2 shows the permitted status transitions. A shipment starts in PENDING; the legal moves are PENDING → IN\_TRANSIT or PENDING → RETURNED, then IN\_TRANSIT → DELIVERED or IN\_TRANSIT → RETURNED. DELIVERED and RETURNED are terminal: every other transition raises `InvalidStatusTransitionException`.



**Figure 2:** Shipment life-cycle state diagram. Any transition not shown raises `InvalidStatusTransitionException` with the exact message "Invalid status transition: <FROM> -> <TO>".

## 4 Implementation

Full source code (nine .java files) is available in the project repository (Appendix D). This section presents the key excerpts that demonstrate how each required OOP concept is realised.

### 4.1 Interface and Abstract Class

```

1 public interface Insurable {
2     double getInsuranceCost();
3 }
  
```

**Listing 1:** Insurable interface (one file, one method).

```

1 public abstract class Shipment implements Insurable {
2     private static int counter = 0;           // shared static ID counter
3
4     protected final int id;
5     protected final String sender, recipient;
6     protected final double distanceKm, weightKg;
7     protected ShipmentStatus status = ShipmentStatus.PENDING;
8
9     protected Shipment(String sender, String recipient,
10                        double distanceKm, double weightKg) {
11         this.id = ++counter;                  // unique sequential ID
  
```

```

12     this.sender      = sender;
13     this.recipient   = recipient;
14     this.distanceKm  = distanceKm;
15     this.weightKg    = weightKg;
16 }
17
18 // Subclass-supplied constants
19 public abstract double getRatePerKm();
20 public abstract double getMaxWeightKg();
21 public abstract String typeLabel();           // "Standard"/"Express"/"SameDay"
22
23 // Shared cost formula -- Template Method
24 public double getCost() { return distanceKm * getRatePerKm(); }
25
26 // Status update delegates to the enum state machine
27 public void advanceStatus(ShipmentStatus next) {
28     if (!status.canGoTo(next))
29         throw new InvalidStatusTransitionException(status, next);
30     this.status = next;
31 }
32
33 @Override
34 public String toString() {
35     return String.format(Locale.US,
36         "ID: %d | [%s] %s -> %s (%.1f km, %.1f kg) | Cost: %.2f TL | Status: %s",
37         id, typeLabel(), sender, recipient,
38         distanceKm, weightKg, getCost(), status);
39 }
40 }

```

Listing 2: Shipment abstract base class (key parts).

The pattern `distanceKm * getRatePerKm()` is an application of the *Template Method* design pattern: the algorithm skeleton is fixed in the base class, while the per-km rate is supplied by each concrete subclass.

```

1 public class SameDayShipment extends Shipment {
2     private static final double RATE_PER_KM    = 6.0;
3     private static final double MAX_WEIGHT_KG  = 10.0;
4     private static final double INSURANCE_PCT  = 0.12;
5
6     public SameDayShipment(String sender, String recipient,
7         double distanceKm, double weightKg) {
8         super(sender, recipient, distanceKm, weightKg);
9     }
10
11     @Override public double getRatePerKm()    { return RATE_PER_KM; }
12     @Override public double getMaxWeightKg()  { return MAX_WEIGHT_KG; }
13     @Override public String typeLabel()      { return "SameDay"; }
14     @Override public double getInsuranceCost() { return getCost() * INSURANCE_PCT; }
15 }

```

Listing 3: SameDayShipment: every subclass follows the same shape.

## 4.2 Enum State Machine

```

1 public enum ShipmentStatus {
2     PENDING, IN_TRANSIT, DELIVERED, RETURNED;
3
4     public boolean canGoTo(ShipmentStatus next) {
5         if (next == null) return false;
6     }
7 }

```

```

6         switch (this) {
7             case PENDING:
8                 return next == IN_TRANSIT || next == RETURNED;
9             case IN_TRANSIT:
10                return next == DELIVERED || next == RETURNED;
11             case DELIVERED:
12             case RETURNED:
13             default:
14                 return false;           // terminal states
15         }
16     }
17 }

```

Listing 4: ShipmentStatus enum with embedded canGoTo.

The enum constant itself is the switch subject, so the compiler guarantees that every constant is considered. Terminal states fall through to `default` and always return `false`, correctly preventing any “resurrection” of a finished shipment.

### 4.3 Custom Exception

```

1 public class InvalidStatusTransitionException extends RuntimeException {
2     private final ShipmentStatus fromStatus, toStatus;
3
4     public InvalidStatusTransitionException(ShipmentStatus from,
5                                             ShipmentStatus to) {
6         super("Invalid status transition: " + from + " -> " + to);
7         this.fromStatus = from;
8         this.toStatus = to;
9     }
10    public ShipmentStatus getFromStatus() { return fromStatus; }
11    public ShipmentStatus getToStatus()   { return toStatus; }
12 }

```

Listing 5: InvalidStatusTransitionException (exact message format).

Extending `RuntimeException` means callers need not declare `throws`. In `Main` the exception is caught at the menu boundary so the program continues:

```

1 try {
2     ShipmentStatus next = ShipmentStatus.valueOf(st);
3     s.advanceStatus(next);
4     System.out.println("Status updated. ID " + id + ": " + prev + " -> " + next);
5 } catch (InvalidStatusTransitionException ex) {
6     System.out.println(ex.getMessage());
7 } catch (IllegalArgumentException ex) {           // unknown enum name
8     System.out.println("Unknown status. Skipping...");
9 }

```

### 4.4 Aggregator with Dual Storage and Comparator Strategies

```

1 public class CargoCompany {
2     private final ArrayList<Shipment> shipments = new ArrayList<>();
3     private final HashMap<Integer, Shipment> shipmentById = new HashMap<>();
4
5     public static final Comparator<Shipment> BY_COST_DESC =
6         (a, b) -> Double.compare(b.getCost(), a.getCost());
7     public static final Comparator<Shipment> BY_DISTANCE_DESC =
8         (a, b) -> Double.compare(b.getDistanceKm(), a.getDistanceKm());
9 }

```

```

9      public static final Comparator<Shipment> BY_STATUS_LIFECYCLE =
10          Comparator.comparingInt(s -> s.getStatus().ordinal());
11
12      public void registerShipment(Shipment s) {
13          if (s.getWeightKg() > s.getMaxWeightKg()) {
14              System.out.println("Weight exceeds capacity for this shipment class. Shipment
15              not registered.");
16              return;
17          }
18          shipments.add(s);
19          shipmentById.put(s.getId(), s);          // both updated atomically
20          System.out.println("Shipment registered. ID: " + s.getId());
21      }
22
23      public Shipment findById(int id) { return shipmentById.get(id); } // O(1)
24
25      public void listSortedBy(String criterion) {
26          Comparator<Shipment> cmp; String header;
27          switch (criterion.trim().toLowerCase(Locale.US)) {
28              case "cost":      cmp = BY_COST_DESC;
29                              header = "==== Sorted by Cost (high -> low) ====="; break;
30              case "distance": cmp = BY_DISTANCE_DESC;
31                              header = "==== Sorted by Distance (high -> low) ====="; break;
32              case "status":    cmp = BY_STATUS_LIFECYCLE;
33                              header = "==== Sorted by Status (PENDING -> RETURNED) =====";
34          }
35          break;
36          default: System.out.println("Unknown option. Skipping..."); return;
37      }
38      List<Shipment> copy = new ArrayList<>(shipments);    // defensive copy
39      copy.sort(cmp);
40      System.out.println(header);
41      for (Shipment s : copy) System.out.println(s);
42  }
43
44      public void summary() {
45          double stdIns = 0, expIns = 0, sdIns = 0;
46          for (Shipment s : shipments) {
47              if (s instanceof StandardShipment) stdIns += s.getInsuranceCost();
48              else if (s instanceof ExpressShipment) expIns += s.getInsuranceCost();
49              else if (s instanceof SameDayShipment) sdIns += s.getInsuranceCost();
50          }
51          System.out.println("==== Revenue & Insurance Summary =====");
52          System.out.println("Total Shipments: " + shipments.size());
53          System.out.printf(Locale.US, "Total Revenue: %.2f TL\n", getTotalRevenue());
54          System.out.printf(Locale.US, "Total Insurance: %.2f TL\n", getTotalInsurance());
55          System.out.printf(Locale.US,
56              "(Standard: %.2f + Express: %.2f + SameDay: %.2f)\n",
57              stdIns, expIns, sdIns);
58      }
59  }

```

**Listing 6:** CargoCompany: dual storage, three Comparator strategies, summary.

Every `registerShipment()` call updates both structures atomically, keeping them consistent. `listSortedBy()` sorts a defensive copy so the original insertion order in the list is preserved.

## 4.5 Menu-Driven Main Loop

`Main.main()` runs a `while (running)` loop, reads one line per iteration, and dispatches to the matching `CargoCompany` method via a `switch`. Unknown options print `Unknown option. Skipping...` (NFR-03) and the loop continues; the loop exits cleanly on option 9 with `Exiting. Goodbye!`.

Numeric parsing is guarded by `try/catch` (`NumberFormatException`) and the status-advance call is wrapped in the two-arm catch shown in Listing 5. The first line of `main()` calls `Locale.setDefault(Locale)` so that every `%.2f` output uses a point as decimal separator regardless of the host locale.

## 5 Testing and Results

### 5.1 Specification Walkthrough

The application was tested against the exact sample I/O given in the project specification. With the inputs “Hızlı Kargo” (company name), three registrations (Standard 250 km / 12 kg, Express 50 km / 5 kg, Same-Day 15 km / 3 kg), one legal status advance (ID 1 to `IN_TRANSIT`), one illegal status advance (ID 2 `PENDING` → `DELIVERED`), and the lookup of a non-existent ID (9999), the produced output matches the specification line-for-line. The cost-sort produces:

$$\underbrace{375.00}_{\text{Standard}} > \underbrace{150.00}_{\text{Express}} > \underbrace{90.00}_{\text{Same-Day}} \quad \text{TL}$$

and the summary line reads **Total Revenue: 615.00 TL**, **Total Insurance: 41.55 TL**, with the per-class breakdown (**Standard: 18.75 + Express: 12.00 + SameDay: 10.80**). The full captured transcript is reproduced verbatim in Appendix A.

### 5.2 Numeric Verification

$$\begin{aligned} \text{Cost}_{\text{Std}} &= 250 \times 1.5 = 375.00 \text{ TL} \\ \text{Cost}_{\text{Exp}} &= 50 \times 3.0 = 150.00 \text{ TL} \\ \text{Cost}_{\text{SD}} &= 15 \times 6.0 = 90.00 \text{ TL} \\ \text{Total Revenue} &= 375.00 + 150.00 + 90.00 = \mathbf{615.00 \text{ TL}} \\ \text{Insurance}_{\text{Std}} &= 0.05 \times 375.00 = 18.75 \text{ TL} \\ \text{Insurance}_{\text{Exp}} &= 0.08 \times 150.00 = 12.00 \text{ TL} \\ \text{Insurance}_{\text{SD}} &= 0.12 \times 90.00 = 10.80 \text{ TL} \\ \text{Total Insurance} &= 18.75 + 12.00 + 10.80 = \mathbf{41.55 \text{ TL}} \end{aligned}$$

### 5.3 Edge-Case Test Results

Table 7 reports the edge-case tests executed in addition to the specification walkthrough.

## 6 Limitations and Future Work

### 6.1 Current Limitations

- **No persistence.** Data is held in memory and lost on exit.
- **Console-only interface.** There is no graphical user interface (an optional GUI is offered as a bonus by the specification).
- **Single session, single user.** Concurrent access is unsupported.
- **No date/time tracking.** Status transitions are not timestamped.
- **No cancellation.** A registered shipment cannot be removed.
- **No search by sender or recipient** beyond manual iteration.

**Table 7:** Edge-case test results

| ID    | Scenario / input                               | Expected behaviour                                                    | Result |
|-------|------------------------------------------------|-----------------------------------------------------------------------|--------|
| EC-01 | Register Same-Day with weight 11 kg (> 10 cap) | Weight exceeds capacity for this shipment class. Shipments not stored | ✓ Pass |
| EC-02 | Non-numeric weight (abc)                       | NumberFormatException caught; Invalid number. Skipping...             | ✓ Pass |
| EC-03 | Non-numeric tracking ID at lookup              | Invalid ID. Skipping...; menu loops                                   | ✓ Pass |
| EC-04 | Lookup non-existent ID (9999)                  | findById returns null; No shipment found with that ID.                | ✓ Pass |
| EC-05 | Illegal transition PENDING → DELIVERED         | Exception caught; Invalid status transition: PENDING -> DELIVERED     | ✓ Pass |
| EC-06 | Illegal transition from terminal DELIVERED     | Exception caught; program continues                                   | ✓ Pass |
| EC-07 | Unknown enum name as new status                | Unknown status. Skipping...                                           | ✓ Pass |
| EC-08 | Unknown main-menu option (e.g. x)              | Unknown option. Skipping...; loop continues                           | ✓ Pass |
| EC-09 | Unknown sort criterion (weight)                | Unknown option. Skipping... in listSortedBy                           | ✓ Pass |
| EC-10 | Sort does not alter underlying list order      | Insertion order restored on list-all                                  | ✓ Pass |
| EC-11 | Sequential auto IDs (3 registrations)          | IDs 1, 2, 3 (single static counter)                                   | ✓ Pass |
| EC-12 | Locale correctness with TR host locale         | Decimal separator stays . (Locale.US forced)                          | ✓ Pass |

## 6.2 Potential Extensions

- **Persistence** via object serialisation or a SQLite/JDBC database.
- **Graphical interface** using Swing (JTable, dialogs); an optional GUI earns bonus marks per the specification.
- **Multi-depot support** as suggested by the project scenario.
- **Billing module** that ingests the summary line items.
- **Real-time tracking** integrating GPS pings into the status update path.
- **Timestamps** using `java.time.LocalDateTime`.

## 7 Conclusion

This project delivered a fully functional, console-based Cargo Shipment Tracker in Java, demonstrating every OOP concept required for EE1004 Project 14. The abstract class and inheritance hierarchy separate shared state and behaviour from type-specific tariffs, so a fourth shipment type could be added without touching the aggregator or the menu. The `Insurable` interface keeps the insurance contract orthogonal to the inheritance chain. Polymorphic dispatch on `getRatePerKm()`, `getInsuranceCost()`, and `typeLabel()` lets the rest of the program treat all shipments uniformly while each type behaves correctly, and encapsulation keeps every object in a consistent state.

The `enum` state machine showed that Java enums can carry methods and enforce invariants typo-proof at compile time; the custom exception showed how typed exceptions propagate context-rich errors without burdening intermediate layers; the dual `ArrayList` + `HashMap` design illustrated the classic space–time trade-off mandated by the specification; and the three

**Comparator** strategies showed how the Strategy pattern turns behaviour into a first-class value. The team gained hands-on experience with the core Java OOP constructs, the collections API, lambda expressions, the **Insurable** interface idiom, and the robustness requirements enforced by the auto-grader.

## References

- [1] Marmara University, Faculty of Engineering, Dept. of EEE, “EE1004 — Java Programming: Project Report Guideline, Group Project, Spring 2025–2026,” Marmara University, Istanbul, 2026.
- [2] —, “EE1004 — Java Programming: OOP Study Guide, Spring 2025–2026,” Marmara University, Istanbul, 2026.
- [3] Oracle Corporation, “Java Platform, Standard Edition 11 API Specification,” 2018, [Online]. Available: <https://docs.oracle.com/en/java/>. [Accessed: June 2026].
- [4] B. Eckel, *Thinking in Java*, 4th ed. Upper Saddle River, NJ: Prentice Hall, 2006.
- [5] J. Bloch, *Effective Java*, 3rd ed. Boston, MA: Addison-Wesley, 2018.
- [6] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.
- [7] Oracle Corporation, “Enum Types — The Java Tutorials,” 2024, [Online]. Available: <https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html>. [Accessed: June 2026].
- [8] —, “The Comparator Interface — The Java Tutorials,” 2024, [Online]. Available: <https://docs.oracle.com/javase/tutorial/collections/interfaces/order.html>. [Accessed: June 2026].

## A Console Transcript (Full Run)

The transcript below was captured by running `java Main` under OpenJDK 11 with the inputs supplied by the specification’s worked example (“Hızlı Kargo” company, three shipments, three status advances, two find queries, one cost-sort, the revenue/insurance summary, and exit).

```
Enter company name: Hızlı Kargo

--- Cargo Menu ---
1. Register a standard shipment
2. Register an express shipment
3. Register a same-day shipment
4. Advance shipment status
5. List all shipments
6. Find shipment by ID
7. Sort shipments
8. Show revenue & insurance summary
9. Exit
Choose an option (1-9): 1
Sender: Mehmet
Recipient: Ayşe
Distance (km): 250
Weight (kg): 12
Shipment registered. ID: 1

--- Cargo Menu ---
1. Register a standard shipment
2. Register an express shipment
3. Register a same-day shipment
4. Advance shipment status
```



```
5. List all shipments
6. Find shipment by ID
7. Sort shipments
8. Show revenue & insurance summary
9. Exit
Choose an option (1-9): 2
Sender: Burak
Recipient: Cem
Distance (km): 50
Weight (kg): 5
Shipment registered. ID: 2

--- Cargo Menu ---
1. Register a standard shipment
2. Register an express shipment
3. Register a same-day shipment
4. Advance shipment status
5. List all shipments
6. Find shipment by ID
7. Sort shipments
8. Show revenue & insurance summary
9. Exit
Choose an option (1-9): 3
Sender: Deniz
Recipient: Ece
Distance (km): 15
Weight (kg): 3
Shipment registered. ID: 3

--- Cargo Menu ---
1. Register a standard shipment
2. Register an express shipment
3. Register a same-day shipment
4. Advance shipment status
5. List all shipments
6. Find shipment by ID
7. Sort shipments
8. Show revenue & insurance summary
9. Exit
Choose an option (1-9): 4
Enter shipment ID: 1
Enter new status (IN_TRANSIT/DELIVERED/RETURNED): IN_TRANSIT
Status updated. ID 1: PENDING -> IN_TRANSIT

--- Cargo Menu ---
1. Register a standard shipment
2. Register an express shipment
3. Register a same-day shipment
4. Advance shipment status
5. List all shipments
6. Find shipment by ID
7. Sort shipments
8. Show revenue & insurance summary
9. Exit
Choose an option (1-9): 4
Enter shipment ID: 2
Enter new status (IN_TRANSIT/DELIVERED/RETURNED): DELIVERED
Invalid status transition: PENDING -> DELIVERED

--- Cargo Menu ---
1. Register a standard shipment
2. Register an express shipment
3. Register a same-day shipment
4. Advance shipment status
5. List all shipments
6. Find shipment by ID
7. Sort shipments
8. Show revenue & insurance summary
9. Exit
Choose an option (1-9): 4
Enter shipment ID: 1
Enter new status (IN_TRANSIT/DELIVERED/RETURNED): DELIVERED
Status updated. ID 1: IN_TRANSIT -> DELIVERED
```

```
--- Cargo Menu ---
1. Register a standard shipment
2. Register an express shipment
3. Register a same-day shipment
4. Advance shipment status
5. List all shipments
6. Find shipment by ID
7. Sort shipments
8. Show revenue & insurance summary
9. Exit
Choose an option (1-9): 5
ID: 1 | [Standard] Mehmet -> Ayşe (250.0 km, 12.0 kg) | Cost: 375.00 TL | Status: DELIVERED
ID: 2 | [Express] Burak -> Cem (50.0 km, 5.0 kg) | Cost: 150.00 TL | Status: PENDING
ID: 3 | [SameDay] Deniz -> Ece (15.0 km, 3.0 kg) | Cost: 90.00 TL | Status: PENDING

--- Cargo Menu ---
1. Register a standard shipment
2. Register an express shipment
3. Register a same-day shipment
4. Advance shipment status
5. List all shipments
6. Find shipment by ID
7. Sort shipments
8. Show revenue & insurance summary
9. Exit
Choose an option (1-9): 6
Enter shipment ID: 2
ID: 2 | [Express] Burak -> Cem (50.0 km, 5.0 kg) | Cost: 150.00 TL | Status: PENDING
Insurance: 12.00 TL

--- Cargo Menu ---
1. Register a standard shipment
2. Register an express shipment
3. Register a same-day shipment
4. Advance shipment status
5. List all shipments
6. Find shipment by ID
7. Sort shipments
8. Show revenue & insurance summary
9. Exit
Choose an option (1-9): 6
Enter shipment ID: 9999
No shipment found with that ID.

--- Cargo Menu ---
1. Register a standard shipment
2. Register an express shipment
3. Register a same-day shipment
4. Advance shipment status
5. List all shipments
6. Find shipment by ID
7. Sort shipments
8. Show revenue & insurance summary
9. Exit
Choose an option (1-9): 7
Sort by (cost/distance/status): cost
===== Sorted by Cost (high -> low) =====
ID: 1 | [Standard] Mehmet -> Ayşe (250.0 km, 12.0 kg) | Cost: 375.00 TL | Status: DELIVERED
ID: 2 | [Express] Burak -> Cem (50.0 km, 5.0 kg) | Cost: 150.00 TL | Status: PENDING
ID: 3 | [SameDay] Deniz -> Ece (15.0 km, 3.0 kg) | Cost: 90.00 TL | Status: PENDING

--- Cargo Menu ---
1. Register a standard shipment
2. Register an express shipment
3. Register a same-day shipment
4. Advance shipment status
5. List all shipments
6. Find shipment by ID
7. Sort shipments
8. Show revenue & insurance summary
9. Exit
Choose an option (1-9): 8
```

```

===== Revenue & Insurance Summary =====
Total Shipments: 3
Total Revenue: 615.00 TL
Total Insurance: 41.55 TL
(Standard: 18.75 + Express: 12.00 + SameDay: 10.80)

--- Cargo Menu ---
1. Register a standard shipment
2. Register an express shipment
3. Register a same-day shipment
4. Advance shipment status
5. List all shipments
6. Find shipment by ID
7. Sort shipments
8. Show revenue & insurance summary
9. Exit
Choose an option (1-9): 9
Exiting. Goodbye!

```

## B Member Contributions

**Table 8:** Member contributions (signed by every group member)

| Member              | Student ID | Components implemented                                                                                                                                          | Approx. |
|---------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| Atabey Aydı         | 150718503  | ShipmentStatus enum and canGoTo(...) state machine; InvalidStatusTransitionException with the exact spec-mandated message.                                      | ≈17 %   |
| Mehmet Açar         | 150719020  | Shipment abstract base class, the static ID counter, advanceStatus(...), and the spec-format toString().                                                        | ≈17 %   |
| İsmail Hanifi Nal   | 150719025  | StandardShipment, ExpressShipment, SameDayShipment subclasses with their rate/cap/insurance constants; the Insurable interface.                                 | ≈17 %   |
| Abdulkadir Köroğlu  | 150719695  | CargoCompany: registerShipment weight-cap guard, listAllShipments, findById (HashMap path), getTotalRevenue/getTotalInsurance.                                  | ≈17 %   |
| Burak Gökmen        | 150720010  | CargoCompany: listSortedBy with the three Comparator strategies (cost / distance / status), and the summary() revenue & insurance breakdown.                    | ≈17 %   |
| Alperen Tufan Pelit | 150720012  | Main menu loop, input validation (try/catch around every parse), the two-arm catch for status advance, the captured console transcript, and report compilation. | ≈17 %   |

*All members reviewed the complete codebase, contributed to debugging and testing, and committed from their own GitHub accounts (see the repository commit history).*

## C OOP Concept Self-Assessment

**Table 9:** OOP concept self-assessment

| Status | Requirement                                                       | Location                                                                                                  |
|--------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| ✓      | Abstract class with $\geq 3$ concrete sub-classes                 | <code>Shipment;</code><br><code>Standard/Express/SameDayShipment</code>                                   |
| ✓      | Interface implemented by every subclass                           | <code>Insurable / getInsuranceCost()</code>                                                               |
| ✓      | Polymorphic, overridden <code>toString()</code> for printing      | <code>Shipment.toString()</code> dispatched on <code>typeLabel()</code>                                   |
| ✓      | Aggregator class with an <code>ArrayList</code> of objects        | <code>CargoCompany.shipments</code>                                                                       |
| ✓      | <code>HashMap</code> for $O(1)$ lookup (mandatory)                | <code>CargoCompany.shipmentById</code>                                                                    |
| ✓      | Constants as <code>private static final</code> (no magic numbers) | <code>RATE_PER_KM</code> , <code>MAX_WEIGHT_KG</code> ,<br><code>INSURANCE_PCT</code>                     |
| ✓      | Robust error handling (no crashes)                                | <code>try/catch</code> in <code>advance/find/parse</code> paths;<br>default switch branches               |
| ✓      | <code>enum</code> status with state machine                       | <code>ShipmentStatus.canGoTo()</code>                                                                     |
| ✓      | Custom exception (unchecked)                                      | <code>InvalidStatusTransitionException</code>                                                             |
| ✓      | Three named <code>Comparator</code> strategies                    | <code>cost</code> , <code>distance</code> , <code>status</code> constants in<br><code>CargoCompany</code> |
| ✓      | Static ID counter                                                 | <code>counter</code> in <code>Shipment</code>                                                             |
| ✓      | <code>@Override</code> on every overriding method                 | all overrides in subclasses                                                                               |
| ✓      | Locale-correct numeric formatting                                 | <code>Locale.US</code> in <code>toString()</code> and<br><code>summary()</code>                           |

## D Pre-Submission Checklist

**Table 10:** Pre-submission checklist

| Item                                                                  | Done? |
|-----------------------------------------------------------------------|-------|
| Cover page complete (members, IDs, project number, instructors, date) | ✓     |
| Abstract and keywords included                                        | ✓     |
| Table of contents included                                            | ✓     |
| UML class diagram present and matches the code                        | ✓     |
| OOP-pillars + interface decoupling table completed                    | ✓     |
| Testing section with spec walkthrough and edge-case table             | ✓     |
| Console transcript included verbatim (Appendix A)                     | ✓     |
| References in IEEE style; all sources cited                           | ✓     |
| GitHub repository public; link and commit hash stated (Appendix D)    | □     |
| Source .zip includes all 9 .java files                                | ✓     |
| Contribution table completed and signed by every member               | ✓     |
| AI-use disclosure included                                            | ✓     |
| Report exported to PDF                                                | ✓     |

## E GitHub Repository Information

**Action required before submission:** complete the fields below with the group’s actual repository URL and the commit hash of the submitted version. Verify the link in a private/incognito window before submitting.

**Table 11:** GitHub repository information

| Field                        | Value                                                           |
|------------------------------|-----------------------------------------------------------------|
| Repository URL               | <code>https://github.com/atabeyaydi/CargoShipmentTracker</code> |
| Commit hash (40 chars)       | [PASTE FULL COMMIT HASH HERE]                                   |
| Branch                       | <code>main</code>                                               |
| Verified in incognito window | □                                                               |

Build and run (for the grader): `javac *.java` then `java Main`, supplying the console inputs shown in Appendix A.

## F AI-Use Disclosure

In accordance with the academic-integrity policy of the EE1004 Project Report Guideline [1], the group discloses the following:

- **Tools used:** an AI programming assistant was used during the development and report-writing phases.
- **Purpose:** drafting the report scaffolding (LaTeX preamble, section structure, table layouts), reviewing the enum state machine and custom-exception design, and proof-reading the implementation chapter.

- **Team understanding:** every member has reviewed, discussed, and understands every line of submitted code, and is prepared to explain their assigned component in an individual viva.
- **Attribution:** all external sources are cited in the References; no code was copied from other groups or uncited web sources.